

RFID Attendance Admin Panel — Full Project Summary

What It Is

A **Single Page Application (SPA)** that serves as the admin dashboard for an RFID-based attendance system. It connects to a **Go backend API** hosted on Render, and lets administrators manage classes, students, view attendance reports, and monitor live scan logs. Deployed on **Vercel**.

Tech Stack (with exact versions)

Technology	Version	Role
React	19.2	UI library — component-based rendering, hooks for state & lifecycle
Vite	7.3.1	Build tool & dev server — uses ESBuild for fast HMR, Rollup for production bundling
Tailwind CSS	4.2.1	Utility-first CSS framework — all styling done via class names, no separate CSS files
React Router	6.30.3	Client-side routing — BrowserRouter, nested Route elements, NavLink for active states
Recharts	3.8.0	Charts — BarChart, PieChart, ResponsiveContainer, Cell, Tooltip, Legend
Lucide React	0.577+	Icon library — SVG icons as React components (Users, GraduationCap, Menu, etc.)
react-hot-toast	2.6.0	Toast notification system — toast.success(), toast.error()
date-fns	4.1.0	Date formatting — format(new Date(), 'yyyy-MM-dd')
ESLint	9.39.1	Code linting — with react-hooks and react-refresh plugins

Architecture & Design Patterns

Component Architecture

- **Functional components** throughout (except ErrorBoundary which is a class component — React requires this)
- **Hooks used:** useState, useEffect, useCallback, useContext, lazy, Suspense
- **Context API** for global auth state (AuthContext wraps the entire app)
- **Custom hook:** useAuth() — returns { apiKey, isAuthenticated, login, logout }

Routing Pattern

- BrowserRouter → single Routes block → nested layout route
- All 7 routes share a common Layout wrapper via Route element={Layout} with Outlet
- Routes: / (Dashboard), /classes, /students, /attendance, /logs, /settings, * (404)

Code Splitting

- Every page is lazy-loaded using React.lazy(() => import('./pages/...')) wrapped in Suspense

- Vite's manualChunks config splits bundles into: vendor (react, react-dom, react-router-dom), charts (recharts), utils (date-fns, lucide-react)

API Layer (src/api.js)

- Single centralized file — all backend calls go through a request() wrapper around fetch()
- Automatic X-API-Key header injection from localStorage
- AbortController with 15-second timeout on every request
- encodeURIComponent on all query params (XSS protection)
- Trailing slash normalization on the base URL

Auth Model

- API key stored in browser localStorage under key rfid_api_key
- Managed via React Context (AuthProvider → useAuth hook)
- Entered by user on the Settings page, persisted across sessions
- No cookies, no sessions, no JWT — just a header-based key

File-by-File Breakdown

Entry & Config Files

File	What It Does
index.html	HTML shell — loads Inter font from Google Fonts, sets theme-color #2563eb, includes noindex meta tag, references /favicon.svg
main.jsx	React entry — createRoot → renders App inside StrictMode
vite.config.js	Vite config — registers react() and tailwindcss() plugins, sets up /api proxy to localhost:8080 for dev, configures manual chunk splitting, disables source maps in prod
vercel.json	Deployment config — SPA catch-all rewrite to index.html, immutable caching on /assets/* (1 year), security headers: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin
.env	Contains VITE_API_URL — baked into the build at compile time via import.meta.env
index.css	Imports Tailwind, defines custom primary color palette (blue shades 50–950) via @theme, custom scrollbar styling, reusable .input class with focus ring

App Shell (App.jsx)

Wraps everything in this order (outside → inside):

1. **ErrorBoundary** — catches any render crash
2. **AuthProvider** — provides API key context
3. **BrowserRouter** — enables client-side routing
4. **Toaster** — dark-themed toast container, top-right, 3s duration
5. **Suspense** — shows LoadingScreen while lazy chunks load
6. **Routes** — nested layout route with all pages

Reusable Components

Layout (Layout.jsx)

Full app shell with:

- Fixed sidebar (w-64) with Radio icon + "RFID Admin" branding
- 6 NavLink items with active state highlighting (bg-primary-50 text-primary-700)
- Mobile hamburger menu with backdrop overlay (hidden on lg: breakpoint)
- Sticky header with green pulse dot ("System Online")
- Outlet for page content

Card, CardHeader, CardBody (UI.jsx)

- **Card** — White rounded-xl container with border and shadow-sm
- **CardHeader** — Top section with bottom border, px-6 py-4
- **CardBody** — Content area with padding px-6 py-4

StatCard (UI.jsx)

- Props: icon, label, value, sublabel, color
- Renders colored icon box (w-12 h-12) + label + large bold value
- 6 color variants: primary, green, red, amber, violet, sky

Badge (UI.jsx)

- Props: children, variant
- Pill-shaped label (rounded-full, text-xs, font-medium)
- 5 variants: default (gray), success (green), danger (red), warning (amber), info (blue)

Button (UI.jsx)

- Props: children, variant, size, className, ...rest (spread to button element)
- 4 variants: primary (blue fill), secondary (white with border), danger (red fill), ghost (transparent)
- 3 sizes: sm, md, lg
- Built-in focus ring, disabled opacity + cursor states

EmptyState (UI.jsx)

- Props: icon, title, description, action
- Centered placeholder with rounded icon circle + heading + optional description + optional action button

Spinner & LoadingScreen (UI.jsx)

- **Spinner** — Animated SVG using CSS animate-spin
- **LoadingScreen** — Centered spinner at h-64, used as Suspense fallback

Modal (Modal.jsx)

- Props: open, onClose, title, children, size
- Fixed overlay — black/50 backdrop, white rounded-xl panel
- Close X button in header
- 4 size variants: sm (max-w-md), md (max-w-lg), lg (max-w-2xl), xl (max-w-4xl)
- Max height 90vh with overflow scroll

ErrorBoundary (ErrorBoundary.jsx)

- **Class component** (React requirement for error boundaries)
- Uses `getDerivedStateFromError` + `componentDidCatch`
- Full-screen error UI with `AlertTriangle` icon and "Try Again" button that resets state

- Logs errors to console
-

Pages

Dashboard (Dashboard.jsx)

- Fetches `/api/stats` on mount, auto-refreshes every 30 seconds via `setInterval`
- **6 StatCards** in responsive grid (1/2/3/6 columns): Total Students, Total Classes, Today's Scans, Present Today, Absent Today, Attendance Rate (%)
- **Bar chart** (Recharts BarChart) — attendance by class, green bars for Present, red for Absent, class names truncated at 12 chars
- **Donut pie chart** (Recharts PieChart with `innerRadius`) — present vs absent breakdown with labels
- **Recent scans table** — student name, roll number, scan time, "Scanned" badge
- Hooks: `useState`, `useEffect`

Classes (Classes.jsx)

- Full **CRUD** — create, read, update, delete classes via modals
- **Card grid** layout (1/2/3 columns responsive) — each card shows class name, department, semester badge, section, room, teacher, student count
- **Capacity progress bar** — horizontal bar showing `student_count / total_capacity` as percentage
- **Search** — client-side filter by class name, department, or teacher name
- **8-field form** in modal: class_name, department, semester (number 1-12), section, room_number, total_capacity, teacher name, teacher email
- `window.confirm()` for delete confirmation
- Helper component: `Field` (label wrapper with required asterisk)

Students (Students.jsx)

- Full **CRUD** with the most complex form (15 fields)
- **Table view** with columns: Roll No., Name (clickable → opens detail modal), UID (monospace), Class, Gender (color-coded badge), Phone, Actions
- **Responsive column hiding**: UID hidden below md, Class/Gender below lg, Phone below xl
- **Search** by name, roll number, or UID (client-side filtering)
- **Class filter** dropdown — re-fetches from API with `class_id` param
- **CSV export** — generates CSV blob, creates temporary anchor element, triggers download
- **Detail modal** (read-only) — 14 detail rows showing all student fields
- **Create/Edit modal** — 15 fields in 3-column grid: UID, Roll Number, Enrollment No., First Name, Last Name, DOB (date picker), Gender (select), Blood Group (select from 8 types), Class (select), Email, Phone, Admission Year, Parent Name, Parent Phone, Emergency Contact
- Uses `Promise.all` to fetch students + classes in parallel
- Helper components: `StudentDetail`, `DetailRow`, `Field`

Attendance (Attendance.jsx)

- **Filter controls**: Class dropdown + Date picker
- **4 StatCards**: Total, Present, Absent, Rate (%)
- **Attendance table**: Roll No., Name, Status (Present/Absent badge), First In time, Last Out time, Scans count — responsive column hiding on md breakpoint
- **Donut pie chart** — present/absent distribution with legend
- **CSV export** — 7 columns: Roll Number, First Name, Last Name, Status, First In, Last Out, Total Scans
- Fetches class list on mount, then loads report when both `classId` and `date` are set
- Hooks: `useState`, `useEffect`

Logs (Logs.jsx)

- **Paginated table** — 50 records per page, offset-based pagination
- Columns: ID (monospace), Roll No., Student name, UID, Date, Time, Delete action
- **Filters:** date picker + UID text search
- **Pagination controls:** Previous/Next buttons, page number display
- Auto-resets offset when filters change
- **CSV export:** ID, Roll Number, Name, UID, Date, Time
- **Delete:** removes all scans for a UID on a specific date (with window.confirm)
- Hooks: useState, useEffect, useCallback (memoized loadLogs)

Settings (Settings.jsx)

- **Server health check** — calls GET / (health endpoint), shows Connected/Unreachable with CheckCircle/XCircle icon, displays server version
- **API Key management:** form to enter key, stores via useAuth().login(), masked display (first 6 chars + dots), "Test Key" button (tries getClasses), "Remove Key" button
- **Configuration display:** shows current VITE_API_URL, environment badge (Development/Production based on import.meta.env.DEV)

NotFound (NotFound.jsx)

- Large "404" text, "Not found" heading, link back to Dashboard

API Communication

All 17 endpoints go through src/api.js:

Method	Endpoint	Used By
GET	/	Settings (health check)
GET	/api/stats	Dashboard
POST	/api/scan	(scan simulation)
GET	/api/attendance/daily?date=&class_id=	Attendance
GET	/api/classes	Classes, Students, Attendance, Settings
GET	/api/class?id=	(single class fetch)
POST	/api/classes	Classes (create)
PUT	/api/class?id=	Classes (update)
DELETE	/api/class?id=	Classes (delete)
GET	/api/students?class_id=	Students
GET	/api/student?id=	(single student)
POST	/api/students	Students (create)
PUT	/api/student?id=	Students (update)

DELETE	/api/student?id=	Students (delete)
GET	/api/logs?limit=&offset=&date=&uid=	Logs
DELETE	/api/log?uid=&date=	Logs (delete)

Styling System

- **Tailwind CSS v4** with @tailwindcss/vite plugin (no PostCSS config needed)
- **Custom theme** defined in index.css via @theme — 11-shade blue primary palette (50 through 950)
- **Font:** Inter (loaded from Google Fonts with preconnect for performance)
- **Custom .input class** for form inputs — 1px gray border, 0.5rem padding, blue focus ring with box-shadow
- **Custom scrollbar** — thin 6px width, slate colors, rounded thumb
- **Responsive breakpoints used:** sm (640px), md (768px), lg (1024px), xl (1280px)
- **No separate CSS modules or styled-components** — everything is Tailwind utility classes inline

Production Optimizations

Optimization	Implementation
Code splitting	React.lazy + Suspense for all 7 routes, plus manualChunks in Vite config splitting vendor/charts/utlis
Asset fingerprinting	Vite hashes all output filenames (e.g., index-DI33VUjr.js) for cache busting
Immutable caching	vercel.json sets max-age=31536000, immutable on /assets/*
No source maps	sourcemap: false in build config — prevents source code exposure in prod
Security headers	X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin
Request timeouts	15s AbortController prevents hanging requests
Error boundary	Catches render crashes, shows retry UI instead of white screen
Font preconnect	link rel="preconnect" to fonts.googleapis.com and fonts.gstatic.com

Build output: 15 chunks, ~5.9s build time, largest chunk is charts at 379KB (112KB gzipped)

React Concepts Used

- useState, useEffect, useCallback, useContext
- createContext, lazy, Suspense, StrictMode
- Class component lifecycle (getDerivedStateFromError, componentDidCatch)
- Context API provider/consumer pattern
- Custom hooks (useAuth)
- Controlled form inputs
- Conditional rendering
- Array .map() for list rendering

- Promise.all for parallel fetches
 - props.children pattern
 - Spread props (...props)
 - Render props via className function in NavLink
-

JavaScript Concepts Used

- async/await
- fetch API
- AbortController
- localStorage
- setTimeout / clearTimeout
- URLSearchParams
- encodeURIComponent
- JSON.stringify / JSON.parse
- Blob + URL.createObjectURL (CSV export)
- document.createElement('a') for download trigger
- Template literals
- Destructuring (object and array)
- Optional chaining
- Nullish coalescing
- Array methods: filter, map, reduce
- import.meta.env (Vite environment variables)
- ES modules (import/export)